STUDENT PERFORMANCE & ATTENDANCE ANALYSIS SYSTEM

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410015	26102000137	BWU/MCA/26/019	Trisha Biswas

in partial fulfillment for the award

of the degree Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

CERTIFICATE

This is to certify that the project report entitled “Library Management System” is submitted by _____ as part of the academic work for the course / programme _____. The project has been prepared under the guidance of the concerned faculty member.

To the best of my knowledge, the work presented in this report describes a Python-based web application for maintaining book, student and issue-return records using CSV files.

Project Supervisor	Head of Department
MR.Rahul Kumar Ghosh	DR. Jayanta Aich
SUPERVISOR	HEADOFTHEDEPARTMENT
Assistant Professor	Computational Sciences
Computational Sciences	Brainware University

DECLARATION

I hereby declare that this project report titled “Library Management System” has been prepared by me for academic purposes. The report explains the design, implementation and working of the application developed using Python, Flask, HTML, CSS, CSV file handling and Matplotlib.

I have used the project files and the sample data available with the application to describe its features. Any external material referred to during preparation is acknowledged in the references section.

ABSTRACT

The Library Management System is a web-based application developed to make routine library activities easier to manage. In a small library, maintaining book details, student information and issue-return records manually can take time and may lead to mistakes. This project provides a simple interface through which a user can add books, register students, search the catalogue, issue books and record returned books.

The application is developed in Python using the Flask framework. The main library operations are organized in a separate library.py module, while app.py connects those operations to web pages. Book, student and transaction information is stored in CSV files, so the records remain available when the application is restarted. The system checks duplicate IDs, verifies whether a book and student exist, and prevents an already-issued book from being issued again.

The issue period is set to 14 days. When a book is returned after its due date, the application calculates an overdue charge of ₹5 for each late day. The dashboard also presents summary information and three Matplotlib visualizations: a bar chart for record counts, a line graph for monthly issue activity and a histogram for book publication years. The project demonstrates how Python programming, web development, file handling and basic data visualization can be combined in one practical application.

The current version is intended as a learning project and uses CSV files rather than a database. It can be extended in the future with login facilities, database storage, report export and more detailed analytics.

TABLE OF CONTENTS

Chapter	Title	Page
1	Introduction	_____
2	Objectives	_____
3	Project Planning	_____
4	Requirement Analysis	_____
5	System Flow and Diagrams	_____
6	Proposed System Design	_____
7	Implementation and Working	_____
8	Testing and Experimental Results	_____
9	Future Scope	_____
10	Conclusion	_____
	References	_____

Note: Page numbers can be updated after the final formatting or printing of this report.

LIST OF FIGURES

- Figure 5.1 Context-Level View of the System
- Figure 5.2 Main System Data Flow
- Figure 5.3 Book Issue and Return Workflow
- Figure 8.1 Dashboard and Summary Cards
- Figure 8.2 Bar Chart
- Figure 8.3 Monthly Issue Line Graph
- Figure 8.4 Publication-Year Histogram

CHAPTER 1: INTRODUCTION

A library contains books and other learning resources that need to be organized properly. The librarian must keep track of which books are available, which students have borrowed them and when the books are expected to be returned. When these records are maintained only in notebooks or separate spreadsheets, searching and updating information can become difficult.

The Library Management System developed for this project provides a browser-based way to perform common library tasks. A user can view the catalogue, add new books, register students, search by title or author, issue a book and record its return. The dashboard gives a quick summary of the library records and displays charts generated from the stored data.

The project uses Flask to handle web requests and render HTML templates. The Library class contains the main rules and data operations. CSV files are used for storing books, students and transactions. Matplotlib is used to create chart images that are displayed on the dashboard.

The main purpose is not to build a large commercial library platform, but to demonstrate a working application with a clear structure and useful features. The project also provides practical experience with Python classes, routes, forms, validation, file handling and visualization.

CHAPTER 2: OBJECTIVES

- To maintain book information such as book ID, title, author, publication year and availability status.
- To store basic student details including student ID, name, email and phone number.
- To provide a search facility that can find books using a title or author keyword.
- To record book issue and return activities with issue date, due date and return date.
- To prevent duplicate book IDs and student IDs and to check book availability before issuing.
- To calculate an overdue charge of ₹5 per late day after the 14-day issue period.
- To show useful library statistics through dashboard cards and graphical charts.
- To keep the application modular so that the web routes and library operations are separated.

These objectives describe the features implemented in the current project version. Features such as user authentication and online database access are not included in this version.

CHAPTER 3: PROJECT PLANNING

3.1 Understanding the problem

The first step was to identify the routine tasks of a small library: maintaining the catalogue, keeping student details, tracking borrowed books and recording returns. The project was planned around these tasks so that each one could be accessed from a web page.

3.2 Choosing the tools

- Python was selected for the main application logic.
- Flask was selected to create routes and display HTML templates in a browser.
- CSV files were selected for simple record storage without setting up a database server.
- Matplotlib was selected to generate the dashboard charts.

3.3 Development sequence

- Prepare the Book, Student and Library classes.
- Create functions for adding, searching, issuing and returning records.
- Connect the library functions with Flask routes and HTML forms.
- Store records in separate CSV files.
- Create dashboard charts from the available records.
- Run the application locally and check the main user operations.

3.4 Project files

The project is organized into app.py, library.py, a data folder containing CSV files, templates for the web pages, and a static folder for CSS and generated chart images. This arrangement keeps application logic, data and presentation files in separate locations.

CHAPTER 4: REQUIREMENT ANALYSIS

4.1 Functional requirements

- Display a dashboard with book, student and transaction summaries.
- List books and filter the list by author.
- Add a book after checking required fields and duplicate ID.
- Register a student after checking required fields and duplicate ID.
- Search books by title or author.
- Issue a book to a registered student if the book is available.
- Return an issued book and calculate any overdue charge.
- Display transaction history and generate dashboard charts.

4.2 Non-functional requirements

- The pages should be simple to navigate in a modern web browser.
- The application should provide clear success or error messages.
- CSV records should be saved with UTF-8 encoding.
- The code should be divided into manageable modules.

4.3 Hardware and software requirements

Type	Requirement
Hardware	Computer with a basic modern processor, memory and storage
Operating system	Windows, Linux or macOS
Language	Python 3
Web framework	Flask
Visualization	Matplotlib
Browser	Any modern web browser

CHAPTER 5: SYSTEM FLOW AND DIAGRAMS

The system follows a simple request-and-response process. The user opens a page in the browser and submits a form when an operation is required. Flask receives the request, calls the appropriate method of the Library class, and the library module validates and updates the records. Updated information is saved in CSV files.

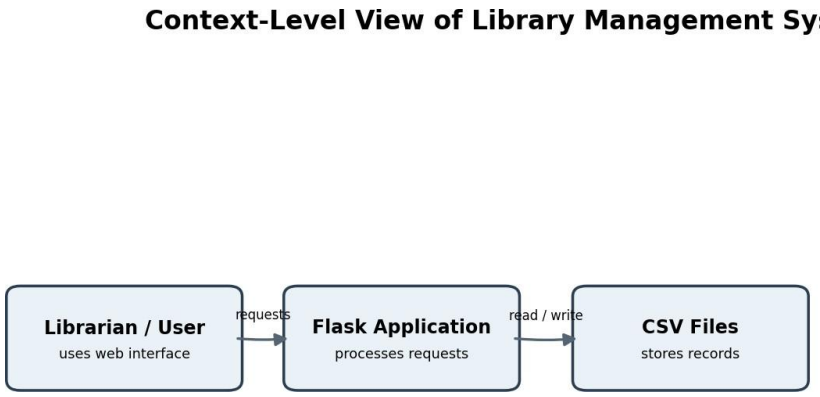


Fig. 5.1: Context-Level View of the Library Management System

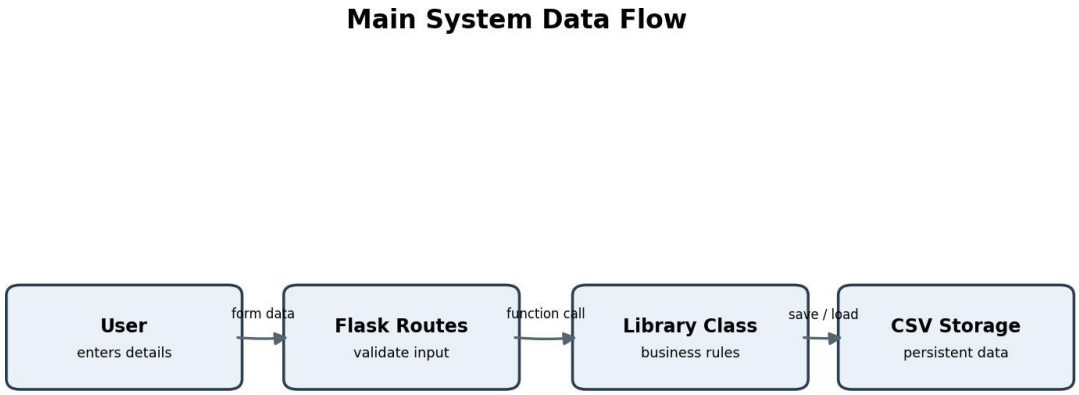


Fig. 5.2: Main System Data Flow

5.1 Book issue and return flow

For issuing a book, the application checks whether the given Book ID and Student ID exist. It also checks that the book is currently available. If the checks pass, the system creates a transaction with today's date and a due date fourteen days later. During return, the system finds the active issue record, calculates late days and updates the transaction and book status.

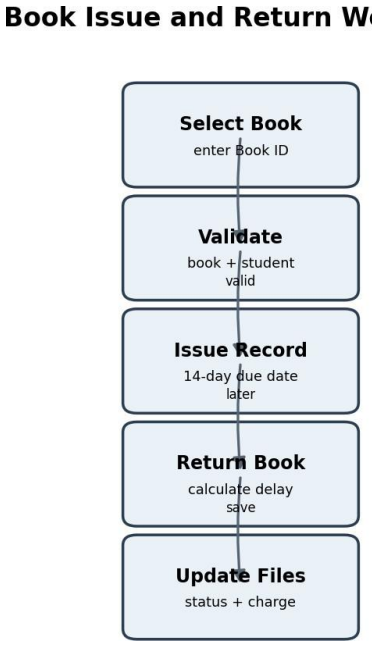


Fig. 5.3: Book Issue and Return Workflow

CHAPTER 6: PROPOSED SYSTEM DESIGN

6.1 Application architecture

The application has three main layers. The presentation layer consists of HTML templates and CSS. The web layer is implemented in app.py using Flask routes. The logic and storage layer is implemented in library.py and the CSV files. Matplotlib chart functions in app.py read the in-memory library records and save chart images in the static directory.

6.2 Main data entities

Entity	Important fields	Purpose
Book	book_id, title, author, year, status	Stores catalogue details
Student	student_id, name, email, phone	Stores borrower details
Transaction	book_id, student_id, issue_date, due_date, return_date, charge, status	Tracks each issue and return

6.3 Data storage

Three CSV files are used: books.csv, students.csv and transactions.csv. The Library class loads these files when the application starts. When a record is added or an issue/return operation is completed, the related save function rewrites the relevant CSV file with the updated records.

6.4 Main business rules

- A Book ID must be unique.
- A Student ID must be unique.
- A book can only be issued when its status is Available.
- The due date is calculated as 14 days after the issue date.
- The overdue charge is ₹5 multiplied by the number of days returned late.

CHAPTER 7: IMPLEMENTATION AND WORKING

7.1 Application setup

The Flask application is created in app.py. The project directory is resolved using pathlib so that the program works with its own data and static folders. The application loads the CSV records into a Library object before serving requests.

7.2 Book and student management

The add-book page collects Book ID, title, author and publication year. The add-student page collects Student ID, name, email and phone number. The Library methods check required fields and duplicate identifiers before saving a new object.

7.3 Search and catalogue

The search page accepts a keyword and calls search_books(). The search is case-insensitive and checks both the title and author fields. The books page also supports filtering by author.

7.4 Issue and return

The issue form accepts a Book ID and Student ID. The application validates both records and checks availability before creating a transaction. The return form accepts a Book ID and updates the latest active transaction for that book. A late return results in a charge of ₹5 per day after the due date.

7.5 Dashboard charts

- Bar chart: compares the number of books, students and transactions.
- Line graph: displays monthly book issue counts using transaction issue dates.
- Histogram: shows the distribution of publication years among books.

The chart functions save PNG files in the static folder. The dashboard uses buttons to switch between the available chart views.

CHAPTER 8: TESTING AND EXPERIMENTAL RESULTS

The application should be tested using the sample CSV records and by entering additional records through the web forms. The following test cases describe the expected behaviour of the implemented features.

Test	Input / Action	Expected result	Status to fill
Add book	Enter a new unique Book ID and complete details	Book is added and appears in catalogue	_____
Duplicate book	Enter an existing Book ID	Error message is displayed	_____
Add student	Enter a new unique Student ID	Student is added to student list	_____
Search	Enter a title or author keyword	Matching books are displayed	_____
Issue book	Use an available book and valid student	Issue transaction is created	_____
Issue unavailable book	Try to issue a book already marked Issued	Operation is rejected	_____
Return book	Return a currently issued book	Book becomes Available	_____
Dashboard	Open the home page	Summary and chart images are displayed	_____

Insert your own screenshots below after running the project. Use screenshots from your actual application so that the report matches the submitted code.

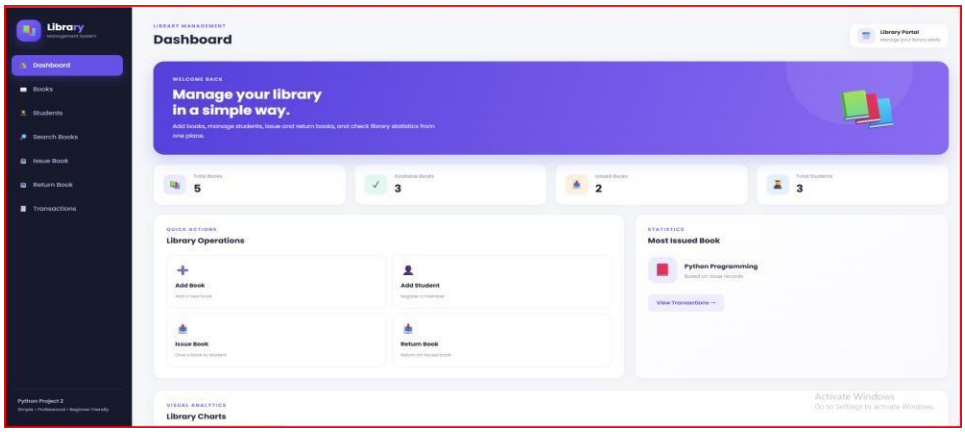


Fig. 8.1: Dashboard showing library summary

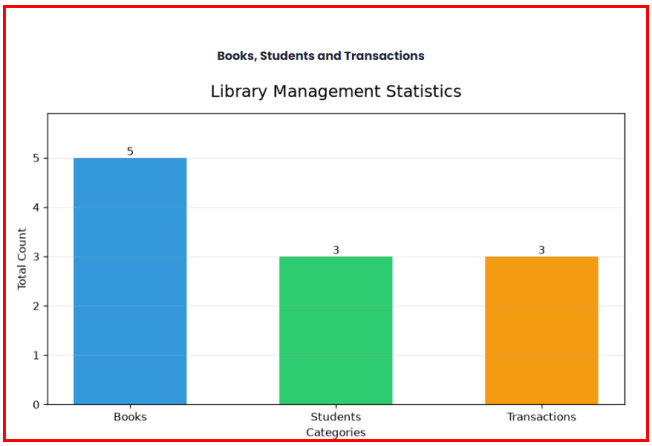


Fig. 8.2: Bar chart of books, students and transactions



Fig. 8.3: Monthly book issue line graph



Fig. 8.4: Histogram of book publication years

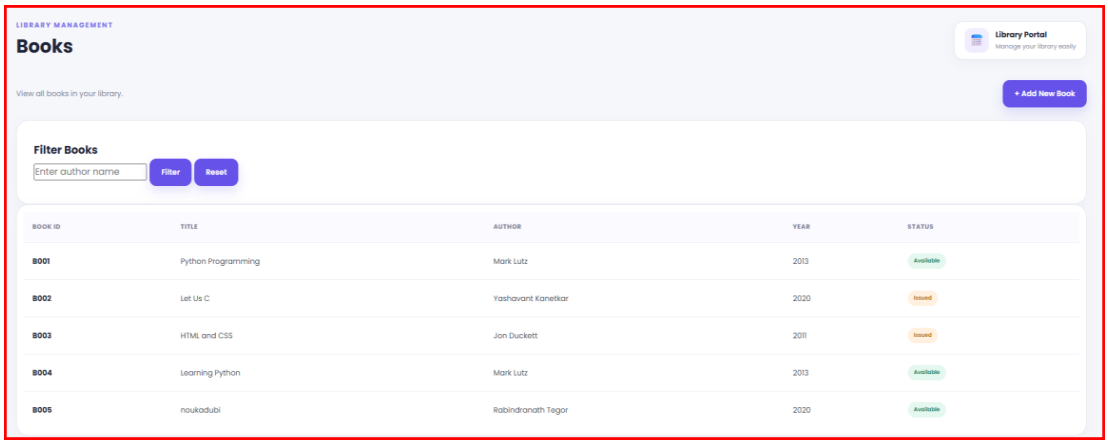


Fig. 8.5: Book issue / return page

CHAPTER 9: FUTURE SCOPE

- Database integration using SQLite or MySQL for stronger data management.
- Login and role-based access for librarian and other users.
- Book cover images and more detailed catalogue information.
- Fine payment tracking and printable receipts.
- Date-wise and student-wise transaction filters.
- Exporting reports to PDF or Excel.
- Backup and restore options for library records.
- Email reminders for books approaching their due dates.

These are possible future improvements and are not claimed as features of the current version.

CHAPTER 10: CONCLUSION

The Library Management System demonstrates how a set of common library activities can be organized in a small web application. It provides facilities to maintain book and student records, search the catalogue, issue books, return books and view transaction history. The use of Flask makes the system accessible through a browser, while the separate Library class keeps the main operations organized.

CSV files provide a simple way to save records, and Matplotlib adds a visual summary through a bar chart, monthly issue line graph and publication-year histogram. The project helped demonstrate practical use of Python classes, form handling, validation, file operations and data visualization.

The current version is suitable as an academic demonstration. It can be improved further by adding a database, user authentication, reporting and backup facilities. Overall, the project meets its basic objective of providing a straightforward tool for managing small-library records.

REFERENCES

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
- Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/
- Python Software Foundation. csv — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html
- Project source files: app.py, library.py, templates/, static/ and data/ (Library Management System project).

Before submission: replace the cover-page blanks, add your institution details, update page numbers if required, and insert screenshots from your own running application. Keep only the test results that you have personally verified.